BuddyPlay Reference Manual v1.0.0

README
BuddyPlay
Stack
Repo layout
Get started
License
BUDDYPLAY-FEATURES
BUDDYPLAY-FEATURES.md — v1.0 feature inventory
Connectivity
Games
Home
Lobby
Local rivalries (instead of leaderboards)
Settings
Telemetry
Monetisation
NOT in v1 (explicitly)
CHANGELOG
Changelog
[Unreleased] - v0.1.0
Commit history (since repo creation)
CONNECTIVITY
CONNECTIVITY.md — BuddyPlay's offline transport layer
§1 — The ladder
§2 — Wire format
§3 — Bonjour / NSD interop
§4 — BLE GATT layout
§5 — Host election
§6 — Per-game transport requirements
§7 — Cross-platform interop matrix
§8 — Pairing UX
§9 — Permissions audit
§10 — Failure modes
DESIGN
BuddyPlay — Design
Pitch
Brand
Repo map
Layered architecture
Per-platform stack
Connectivity ladder
Telemetry
Backend

Domain model
Why no Wi-Fi Direct?
Why no foreground service / background mode?
OBSERVABILITY
OBSERVABILITY.md
v1: nothing
What still exists in code
v1.1 plan (NOT in scope for v1)
Local-only logs
Production debugging without telemetry
PRIVACY
PRIVACY.md
TL;DR
What we collect
What we store on your phone
What we share with other devices
Permissions explained
Children
Changes to this policy
Contact
PRODUCTION
PRODUCTION.md
§1 — No backend
§2 — Storage
§3 — Permissions
§4 — Crash + perf
§5 — Battery & data
§6 — Monetisation
§7 — App size budget
§8 — Cross-platform play matrix
§9 — Watch tier
§10 — Push / background
QUICKSTART
QUICKSTART
Prereqs
60-second iOS spin-up (macOS)
60-second Android spin-up
Pairing two real devices
Dev workflow
Common gotchas
RELEASE-VIDEO-SCRIPT
Release Video Script — BuddyPlay
Hero — top of marketing site | 4
Features overview | 5
Highlight reel | 5
What’s next | 5
Call to action — bottom of page | 4

RELEASING
RELEASING.md
0. Pre-flight
1. Bump version
2. Commit + tag
3. What `release.yml` does
4. Required secrets
5. Pre-release tags
6. Hotfix
7. Local dry-run
SENTRY
SENTRY.md
TL;DR — Sentry is NOT enabled in v1.
Why we kept the stub
v1.1 wiring plan (NOT in scope for v1)
What stays no-op forever
Why not in v1
Where the no-SDK choice is enforced
SIGNING
Signing — BuddyPlay
Android
iOS
Desktop
watchOS
STORE-PACKAGING
STORE-PACKAGING.md
§1 — App Store (iPhone)
§2 — Google Play (Android)
§3 — Marketing assets
§4 — What's new
§5 — Bundle size budget
§6 — Why no Watch / Wear tier
Desktop binaries (Electron)
TESTING
TESTING.md
Shape of the test suite
The keystone tests (these are the project's safety net)
Running locally
Manual cross-platform / cross-transport checklist (pre-release)
Coverage targets

AMERICAN GROUP LLC

BuddyPlay

Reference Manual

Version 1.0.0

Generated 2026-05-08

README

README.md

BuddyPlay

> **Play anywhere. No internet needed.**

BuddyPlay is a lightweight, all-in-one **offline** multiplayer hub. Two phones nearby — by Wi-Fi, mobile hotspot, or Bluetooth LE — pair and play together. No accounts, no servers, no internet required. The "subway tunnel works" experience is the USP.

	iPhone	Android
Royal Chess (turn-based)	✅	✅
Dice Kingdom (Ludo-style)	✅	✅
Mini Racer (real-time twitch)	✅ Wi-Fi/Hotspot	✅ Wi-Fi/Hotspot
Local rivalries store	✅	✅
Connectivity ladder (Wi-Fi → Hotspot → BLE)	✅	✅
Cross-platform play (iOS ↔ Android)	✅	✅

Stack

- **Apple**: Swift / SwiftUI, iOS 17, XcodeGen.
- **Android**: Kotlin / Jetpack Compose, multi-module Gradle (`:core` + `:app`), Hilt, DataStore.
- **Backend**: **None.** Offline-first. No accounts, no servers, no analytics.
- **Shared logic**: `BuddyCore` Swift Package (Apple) and `:core` JVM Kotlin module (Android). Same models, mirrored case-for-case tests.

Repo layout

```
shared/BuddyCore # Apple-side shared models + domain helpers + tests
ios/             # iPhone app
android/         # :core JVM + :app phone
.github/workflows# 6 workflows: ci, ios, android, marketing, pre-release-tests, release
scripts/         # local dev helpers (test-all, run-*-emulator, bump-version)
distribution/    # release whatsnew text per locale
```

See `DESIGN.md` for the full architecture, `CONNECTIVITY.md` for the failover ladder + wire format, and `BUDDYPLAY-FEATURES.md` for the feature inventory.

Get started

```
## iOS Simulator
./scripts/run-ios-sim.sh

## Android emulator
./scripts/run-android-emulator.sh

## Run every test suite
./scripts/test-all.sh
```

License

MIT — see LICENSE.

BUDDYPLAY-FEATURES

BUDDYPLAY-FEATURES.md

BUDDYPLAY-FEATURES.md — v1.0 feature inventory

Connectivity

- ☑ Local Wi-Fi via NSD/Bonjour (`_buddyplay._tcp`) + length-prefixed TCP framing.
- ☑ Mobile Hotspot via the same Wi-Fi path once both peers join.
- ☑ Bluetooth LE GATT fallback on service `0xBP01` (turn-based games only).
- ☑ Auto failover ladder (Wi-Fi → Hotspot → BLE).
- ☑ User-overridable transport choice (auto / Wi-Fi-only / BLE-only).
- ☑ 4-character pairing code (`BUDD-7Q2K`) for visual confirm.
- ☑ Cross-platform iOS ↔ Android peers.
- ☑ Schema-versioned wire format (`v: 1`); decoder rejects unknown versions.
- ☑ Deterministic host election (no round-trip).

Games

- ☑ **Royal Chess** — 8×8, full move legality (castling, en-passant, promotion, check/mate/stalemate). Turn-based, BLE-OK.
- ☑ **Dice Kingdom** — Ludo-style 4-token race. 2-player (DuoPlay). Turn-based, BLE-OK.
- ☑ **Mini Racer** — top-down 2D racer. 30 Hz host tick + client prediction. Wi-Fi or Hotspot only; BLE refused.

Home

- ☑ Horizontal “card scroller” of the 3 games.
- ☑ Persistent floating **Join Nearby Game** button (auto-scans every ~5 s).
- ☑ Tabs: All games · DuoPlay (2P) · Party (3-4P, dimmed in v1).
- ☑ “Last played” carousel above the grid.

Lobby

- ☑ **Host**: pick a game → advertise on Wi-Fi + BLE simultaneously → show pairing code.
- ☑ **Join**: live scan list → tap a peer → confirm code → exchange names → start.

Local rivalries (instead of leaderboards)

- ☑ Per-opponent win/loss/draw tally, per game.
- ☑ Keyed by stable opponent UUID + display name.
- ☑ On-disk JSON store (no cloud).
- ☑ Settings → Erase all rivalries.
- ☑ Settings → Reset device ID.

Settings

- [x] Connectivity preference (auto / Wi-Fi-only / BLE-only).
- [x] Display name.
- [x] Default game.
- [x] Sound on/off.
- [x] Haptics on/off.
- [x] Theme: system / light / dark.
- [x] Telemetry: nothing in v1 (disclosure label).

Telemetry

- [x] Stub interfaces (`AnalyticsService` , `CrashReportingService`) — no transports attached.
- [ ] Real Sentry / PostHog wiring (v1.1).

Monetisation

- [x] `PremiumGateService` stub returning `unlocked = true` .
- [ ] RevenueCat / Play Billing (v1.1).
- [ ] Cached AdMob (v1.1).

NOT in v1 (explicitly)

- [ ] Wi-Fi Direct (`WifiP2pManager`).
- [ ] 3-4 player party games.
- [ ] In-app purchases / ads.
- [ ] Voice chat (v2).
- [ ] UGC / custom maps (v2).
- [ ] Tournament mode (v2).
- [ ] Apple Watch / Wear OS surfaces.
- [ ] Backend / server.
- [ ] Telemetry SDKs.
- [ ] Cloud leaderboards.
- [ ] Push notifications.
- [ ] Foreground service.

CHANGELOG

CHANGELOG.md

Changelog

All notable changes to this project will be documented in this file.

This project adheres to Semantic Versioning.

[Unreleased] - v0.1.0

Initial public release. See distribution/whatsnew/v0.1.0/en-US.txt for the user-facing summary.

Commit history (since repo creation)

- 2719d61 Phase 0: repo skeleton, 11 docs, marketing site, 6 workflows (Srikanth Patchava, 2026-05-06)
- 5dbba75 Phase 1: shared/BuddyCore Swift Package (models, keystone helpers, transports, tests) (Srikanth Patchava, 2026-05-06)
- 222f68c Phase 2: Android :core (Kotlin/JVM mirror of BuddyCore + JUnit tests) (Srikanth Patchava, 2026-05-06)
- 9bbe1c4 Phase 3: iOS phone shell (Home/Lobby/Settings/Rivalries + ConnectivityService + Plist + tests) (Srikanth Patchava, 2026-05-06)
- 8cb15d9 Phase 4: iOS Royal Chess (ChessViewModel/BoardView/Screen) (Srikanth Patchava, 2026-05-06)
- 9e8fe65 Phase 5: iOS Dice Kingdom (LudoViewModel/BoardView/Screen) (Srikanth Patchava, 2026-05-06)
- 26ae5c8 Phase 6: iOS Mini Racer (RacerViewModel/CanvasView/Screen + BLE rejection) (Srikanth Patchava, 2026-05-06)
- 161a21c Phase 7: Android :app shell (manifest, Compose Home/Lobby/Rivalries/Settings, Hilt) (Srikanth Patchava, 2026-05-06)
- 5d10dc2 Phase 8: Android connectivity adapters (NsdDiscovery, WifiTcpTransport, BleTransport, ConnectivityViewModel) (Srikanth Patchava, 2026-05-06)
- 5b1f917 Phase 9: Android Chess + Ludo + Racer Compose screens (Srikanth Patchava, 2026-05-06)
- d0b7b9a fix(core): repair Kotlin syntax error in GameSession.kt (Srikanth Patchava, 2026-05-06)
- 4cd785f fix(app): add material AAR for XML Theme.Material3 parent (Srikanth Patchava, 2026-05-06)
- eff8534 fix(app): annotate startHosting return type as Unit to satisfy override (Srikanth Patchava, 2026-05-06)
- 13d9d39 fix(app): rename SettingsViewModel mutators to avoid JVM setter clash (Srikanth Patchava, 2026-05-06)
- 7ec2333 fix(app): wire Hilt instrumented test infra (runner + deps) for smoke test (Srikanth Patchava, 2026-05-06)
- 0b2459a fix(app): make Settings scrollable; smoke test uses assertExists for off-screen items (Srikanth Patchava, 2026-05-06)
- b74343f fix(app): use onAllNodesWithText().assertCountEquals(1) for off-screen smoke checks (Srikanth Patchava, 2026-05-06)
- db50b6b docs(distribution): add SUBMISSION-CHECKLIST.md (Phase 3 production readiness) (Srikanth Patchava, 2026-05-06)

CONNECTIVITY

CONNECTIVITY.md

CONNECTIVITY.md — BuddyPlay's offline transport layer

This is the load-bearing doc for v1. Two phones must find each other and exchange game state with no internet. The ladder below is the user-visible **Connect** screen; it tries each rung top-down and surfaces a clear status, and lets the user explicitly pick a rung if the auto-detect picks the wrong one.

§1 — The ladder

Rung	Transport	Discovery	Why this order
1	**Local Wi-Fi**	Bonjour / NSD on `_buddyplay._tcp`	Highest throughput, lowest latency, zero user setup.
2	**Mobile Hotspot**	same Bonjour / NSD path once both peers are on the hotspot's subnet	Works when there's no router. Requires user to enable hotspot manually (no programmatic toggle on either platform).
3	**Bluetooth LE**	GATT advertisement on service UUID `0xBP01`	Last resort. Fine for turn-based games (Chess, Ludo). Mini Racer **rejects** this rung.

§2 — Wire format

JSON on Wi-Fi, length-prefixed framing. CBOR opt-in via flag for v1.1.

```
4-byte big-endian length  ‖  utf-8 JSON payload
```

Frame envelope (every payload)

```
{
  "v": 1,
  "sessionId": "<uuid>",
  "from": "<peer-uuid>",
  "kind": "input" | "state" | "lobby" | "ping" | "pong",
  "ts": 1735689600000,
  "payload": { ... }
}
```

Versioning

Decoder hard-fails on unknown major versions and surfaces an **"Update your friend's app"** toast. Tested in `WireCodecTests` / `WireCodecKtTest`.

§3 — Bonjour / NSD interop

Both peers should *publish* their own service AND *scan* for peers. Android's `NsdManager` sometimes skips registrations made before the service was fully initialised; mitigate by running the publish-then-scan sequence with a 1-second debounce + retry loop.

	iOS	Android
Service type	`_buddyplay._tcp`	`_buddyplay._tcp`
Publisher API	`NWListener`	`NsdManager.registerService`
Scanner API	`NWBrowser`	`NsdManager.discoverServices`
Permission prompt	`NSLocalNetworkUsageDescription` (Local Network) — surfaced on first Host/Join action, not at app launch.	`NEARBY_WIFI_DEVICES` (API 33+) or `ACCESS_FINE_LOCATION` (API ≤30) — gated by `usesPermissionFlags="neverForLocation"` for `BLUETOOTH_SCAN` on API 31+.

§4 — BLE GATT layout

Single custom service exposing two characteristics:

Characteristic	UUID	Direction	Properties	Purpose
Inbound (client→host)	`42554450-0001-1000-8000-00805F9B34FB`	client writes	`WRITE_WITHOUT_RESPONSE`	Guest sends inputs to host.
Outbound (host→client)	`42554450-0002-1000-8000-00805F9B34FB`	host notifies	`NOTIFY`	Host streams state back.

Service UUID: `42554450-0000-1000-8000-00805F9B34FB` (the `42554450` prefix spells `BUDP` in ASCII).

Frames > MTU (typically 185 bytes) are split into 4-byte length-prefixed chunks; receiver reassembles. Same envelope as Wi-Fi.

§5 — Host election

Both peers run `HostElection` locally on the `(peerA.id, peerB.id)` tuple. Lexicographically smaller UUID wins; tie-break by platform: iOS wins (no particular reason, just deterministic). No round-trip required — both peers agree without negotiating.

§6 — Per-game transport requirements

Game	Wi-Fi	Hotspot	BLE
Royal Chess	✅	✅	✅
Dice Kingdom	✅	✅	✅
Mini Racer	✅	✅ (RTT-gated)	❌ refused

Mini Racer needs a 30 Hz state tick from host to guest; BLE's GATT throughput + ~30-200 ms latency makes that infeasible. Lobby refuses to start Racer on BLE and recommends Chess / Ludo. On Hotspot, lobby runs an RTT probe and refuses Racer if RTT > 250 ms.

§7 — Cross-platform interop matrix

Host	Guest	Wi-Fi	Hotspot	BLE
iOS	iOS	✅	✅	✅
iOS	Android	✅	✅	✅
Android	iOS	✅	✅	✅
Android	Android	✅	✅	✅

Cross-platform is a hard constraint. Wire format is JSON to make debugging easy across language runtimes.

§8 — Pairing UX

1. **Host** picks a game → BuddyPlay starts advertising on Wi-Fi + BLE simultaneously. UI shows a 4-character pairing code (`BUDD-7Q2K`) for the guest to confirm.
2. **Join** scans the list of advertised peers. Tapping a peer pulls up the same code-confirm step.
3. After confirm, the lobby exchanges player names, runs `HostElection`, and starts the chosen game.

Pairing codes are 4 base32 characters derived from the host's session UUID. Collisions are deliberately allowed (4 chars = 1024 codes) — the user must visually confirm both peers see the same code before proceeding.

§9 — Permissions audit

iOS

- `NSBluetoothAlwaysUsageDescription` — BLE advertise + scan.
- `NSLocalNetworkUsageDescription` — NWBrowser / NWConnection.
- `NSBonjourServices` — declares `_buddyplay._tcp`.

No camera, no microphone, no location. No background modes in v1 (both peers must keep BuddyPlay foregrounded during play).

Android

- `BLUETOOTH_SCAN` + `BLUETOOTH_CONNECT` + `BLUETOOTH_ADVERTISE` (API 31+).
- `BLUETOOTH` + `BLUETOOTH_ADMIN` (API ≤30 fallback).
- `ACCESS_FINE_LOCATION` (only API ≤30, gated by `usesPermissionFlags="neverForLocation"` from API 31).
- `NEARBY_WIFI_DEVICES` (API 33+).
- `ACCESS_WIFI_STATE` + `CHANGE_WIFI_STATE`.
- `INTERNET` (BLE peripheral SDK sometimes wants it during pairing).

No camera, no microphone, no foreground service, no notifications.

§10 — Failure modes

Failure	Detection	Recovery
Wi-Fi router blocks mDNS	Bonjour scan returns empty after 5 s	UI offers “switch to Hotspot” + “switch to BLE”
iOS Local Network prompt rejected	NWBrowser returns `unsatisfied`	UI shows “Enable Local Network in Settings → Privacy” deep link
BLE permission rejected	scanner throws	UI shows “Enable Bluetooth permission in Settings” deep link
Connection drops mid-game	TCP/GATT connection error	UI offers “Reconnect” + auto-saves last move so the rivalry tally still increments on Chess / Ludo
Schema version mismatch	WireCodec decode fails on `v` field	UI shows “Update your friend’s app” toast
RTT > 250 ms on Hotspot Racer	RTT probe before game start	Lobby refuses Racer + recommends Chess / Ludo

DESIGN

DESIGN.md

BuddyPlay — Design

Pitch

BuddyPlay is an **offline multiplayer mini-games hub**. Two phones nearby discover each other over Wi-Fi, Mobile Hotspot, or Bluetooth LE — no internet, no accounts, no servers. v1 ships three games: **Royal Chess**, **Dice Kingdom** (Ludo-style), and **Mini Racer**. The 2-player sub-mode is named **DuoPlay**.

Brand

- **App name**: BuddyPlay
- **Tagline**: *Play anywhere. No internet needed.*
- **Sub-mode for 2P games**: DuoPlay
- **Palette**: deep slate (`#0E1A2B`) + warm coral accent (`#FF6F61`).

Repo map

```
shared/BuddyCore/          Swift Package: models, domain helpers, connectivity
                           adapters, storage, observability stubs, tests.
ios/                       SwiftUI iPhone app (XcodeGen-generated project).
android/                   Multi-module Gradle build:
                             :core    pure-Kotlin/JVM mirror of BuddyCore.
                             :app     Compose phone app + Android-API-bound
                                      connectivity adapters.
.github/workflows/         6 CI workflows. No backend.
scripts/                   Local helpers (test-all, run-*-emulator, bump-version,
                           release-dry-run).
distribution/whatsnew/     Per-locale Play Store whatsnew text.
index.html, styles.css,    Marketing one-pager.
script.js, robots.txt,
sitemap.xml
```

Layered architecture

```
UI                  iOS SwiftUI                Android Compose
                    ──────────                 ───────────────
                    HomeScreen                 HomeScreen
                    Lobby (Host/Join)          Lobby
                    Chess / Ludo / Racer       Chess / Ludo / Racer
                    Rivalries / Settings       Rivalries / Settings
──────────────────────────────────────────────────────────────────
App services        ConnectivityService        ConnectivityViewModel
                    GameSessionService         GameSessionService
                    SettingsModel              SettingsRepo
──────────────────────────────────────────────────────────────────
Shared domain       BuddyCore (Swift)          :core (Kotlin/JVM)
(mirrored 1:1)      ─────────────────          ──────────────────
                    Models                     Models
                    HostElection               HostElection
                    WireCodec                  WireCodec
                    GameStateReducer           GameStateReducer
                    ChessRules                 ChessRules
                    LudoRules                  LudoRules
                    RacerPhysics               RacerPhysics
                    LocalRivalryStore          LocalRivalryStore
                    DeviceIdProvider           DeviceIdProvider
──────────────────────────────────────────────────────────────────
Connectivity        WifiTransport (NW.Socket)  WifiTcpTransport
adapters            BleTransport (CoreBT)      BleTransport (GATT)
                    DiscoveryService (Bonjour) NsdDiscovery (NSD)
                    HotspotAdvisor             HotspotAdvisor
```

The shared-domain layer is pure: no platform APIs, fully unit-tested. Every keystone helper has an XCTest **and** a JUnit twin so behaviour cannot drift between the two implementations.

Per-platform stack

	iPhone	Android
Language	Swift 5.9	Kotlin 1.9
UI framework	SwiftUI	Jetpack Compose
Min OS	iOS 17	API 26 (Android 8)
Project generator	XcodeGen (`ios/project.yml`)	Gradle (`build.gradle.kts`)
DI	manual (env objects)	Hilt
Storage	`UserDefaults` + JSON-on-disk	DataStore + JSON-on-disk
Local network	`Network.framework` (NWBrowser/NWConnection)	`NsdManager` + `Socket`
Bluetooth	`CoreBluetooth`	`BluetoothGattServer` + `BluetoothLeScanner`
Build automation	`xcodebuild` via scripts/CI	`./gradlew` via scripts/CI

Connectivity ladder

See `CONNECTIVITY.md` for the full spec. Summary: try Wi-Fi first (NSD/Bonjour + raw TCP), then Mobile Hotspot (same path), then BLE GATT fallback. Mini Racer rejects BLE and surfaces a “needs Wi-Fi or Hotspot” toast.

Telemetry

None in v1. No Sentry, no PostHog, no analytics SDKs are pulled in. `canImport`-gated stubs exist (`AnalyticsService` , `CrashReportingService`) so v1.1 can opt in trivially without an architecture change.

Backend

None. All state is local. The `LocalRivalryStore` is the only persistent data; it lives in a single JSON file in the app’s documents directory.

Domain model

Type	Notes
`Peer`	UUID, displayName, platform, lastSeenAt.
`GameKind`	enum: `chess`, `ludo`, `racer`. Extensible.
`GameSession`	id, kind, host, guest, transport, startedAt.
`Rivalry`	opponentId, opponentName, perGame: `[GameKind: Record]`.
`WireFrame`	versioned envelope (`v: 1`) wrapping every payload.

Why no Wi-Fi Direct?

Android-only API (`WifiP2pManager`); brittle interop. Mobile Hotspot covers the same use case with a clean cross-platform path. Revisit in v2.

Why no foreground service / background mode?

v1 is a same-room synchronous experience: app is always foregrounded during play. No need to maintain connections in the background. v1.1 may add a background-resume hint when both peers are still on the same Wi-Fi.

OBSERVABILITY

OBSERVABILITY.md

OBSERVABILITY.md

v1: nothing

By design. BuddyPlay is offline-first; we have no server to receive metrics and no consent flow yet for sending data home. The user gets an explicit guarantee:

> **BuddyPlay does not send any data to us.**

This appears in the Settings screen and on the marketing site.

What still exists in code

The `canImport`-gated stubs from the Drift / Card scaffold are kept so v1.1 can opt in trivially without an architecture refactor:

- `shared/BuddyCore/Sources/BuddyCore/Observability/AnalyticsService.swift`
- `shared/BuddyCore/Sources/BuddyCore/Observability/CrashReportingService.swift`
- `android/core/src/main/ … /observability/AnalyticsService.kt`
- `android/core/src/main/ … /observability/CrashReportingService.kt`

All four implement a no-op interface in v1. The methods exist; they do nothing. No SDK is imported.

v1.1 plan (NOT in scope for v1)

If we ever want metrics, the smallest possible step is:

1. Add a Settings toggle: **"Help improve BuddyPlay (sends anonymous game counts; never opponent names)"**. Default OFF.
2. Behind that toggle, attach **PostHog** (open-source, self-hostable) or **Sentry** (crash only).
3. Whitelist: `game_started(kind, transport)`, `game_finished(kind, durationSec)`, `crash(stack)`. Nothing else.
4. Update `PRIVACY.md` to call out the new optional flow.

Local-only logs

Both apps log via the platform-native logger: - iOS: `os_log` / `Logger`. Visible in Console.app while a Sim or device is attached. - Android: `android.util.Log`. Visible via `adb logcat`.

No log persistence. No log forwarding. No file rotation.

Production debugging without telemetry

Until v1.1 attaches a real backend, our debugging story is:

1. **Repro locally** with two devices on the same Wi-Fi.
2. `adb logcat -s BuddyPlay` for Android.
3. **Console.app filtered to** `com.americangroupllc.buddyplay` for iOS.
4. **Reproduce on Simulator** for Wi-Fi/BLE issues — Simulator's BLE works between two simulator instances.
5. Ask users to **send a screen recording**; we don't request crash logs.

PRIVACY

PRIVACY.md

PRIVACY.md

TL;DR

BuddyPlay does not send any data to us. No accounts, no servers, no analytics, no crash reports, no ads. v1 is fully offline.

What we collect

Nothing.

What we store on your phone

Two small JSON files: - `rivalries.json` — your win/loss/draw counts per opponent, keyed by the opponent's stable per-device UUID and their chosen display name. - `device.json` — a random UUID generated once on first launch. Used so other BuddyPlay phones can recognise yours across game sessions.

Both files live in the app's private documents directory. They are never backed up to a server. You can wipe them at any time: - **Settings → Erase all rivalries** clears `rivalries.json`. - **Settings → Reset device ID** regenerates `device.json` (other phones will see you as a brand-new opponent). - **Uninstalling BuddyPlay** removes both files.

What we share with other devices

Only what's required for two phones to play together: - Your **display name** (you set this in Settings). - Your **device UUID**. - **Game state** (whose turn, board position, dice roll, etc.) for the duration of one match.

This data goes only to the other phone you're playing with — never to a server, never to us. It travels over your Wi-Fi network or directly via Bluetooth.

Permissions explained

Permission	What we do with it
Bluetooth	Discover and connect to a nearby BuddyPlay phone when Wi-Fi is unavailable.
Local Network (iOS)	Discover nearby BuddyPlay phones via Bonjour.
Nearby Wi-Fi devices (Android 13+)	Same purpose, modern Android replacement for the legacy location-permission BLE scan.
Location (Android ≤12)	**Not used for location.** Required by Android ≤12 in order to scan for BLE devices. We tag the permission with `neverForLocation` from API 31+ to skip the prompt entirely.

We never request: camera, microphone, contacts, photos, health, motion, HealthKit, location-on-modern-Android, push notifications, calendar.

Children

BuddyPlay is rated 4+ / Everyone. No chat, no UGC, no in-app purchases, no ads. Suitable for minors. We collect no data, so no parental consent flow is required.

Changes to this policy

If a future version (v1.1+) adds optional analytics or crash reporting, this document will be updated, and the new collection will be **opt-in** behind a Settings toggle defaulting to OFF, with a fresh in-app disclosure on the first launch after the update.

Contact

For questions: open an issue at https://github.com/AmericanGroupLLC/BuddyPlay

PRODUCTION

PRODUCTION.md

PRODUCTION.md

How BuddyPlay v1 behaves in the wild.

§1 — No backend

There is no server. There are no accounts. There is no analytics pipeline. v1 is fully offline.

Implications: - We have no way to deprecate old clients except by relying on the `WireCodec` major-version mismatch toast ("Update your friend's app"). - We have no remote-config kill switch. Bugfixes ship as a normal app update on the App Store / Play Store. - We have no telemetry on crashes or feature use. v1.1 will optionally attach Sentry behind a privacy-explicit opt-in toggle in Settings.

§2 — Storage

Single JSON file in the app's documents directory:

Path	iOS	Android
`rivalries.json`	`Documents/buddyplay/rivalries.json`	`filesDir/buddyplay/rivalries.json`
`device.json`	`Documents/buddyplay/device.json`	`filesDir/buddyplay/device.json`

Both files are tiny (single-digit kilobytes even after months of play). We do not encrypt them; an attacker with file-system access has access to your phone anyway. Data is opaque integers (win/loss tallies) and opponent UUIDs — no PII.

§3 — Permissions

Permission	iOS	Android	Why
Bluetooth	`NSBluetoothAlwaysUsageDescription`	`BLUETOOTH_*` group	BLE fallback transport.
Local network	`NSLocalNetworkUsageDescription` + `NSBonjourServices`	`NEARBY_WIFI_DEVICES` (33+) or `ACCESS_FINE_LOCATION` (≤30)	NSD/Bonjour scan.
Camera	—	—	**Not requested.**
Microphone	—	—	**Not requested.**
Notifications	—	—	**Not requested.**
Location	—	API ≤30 only	Required by legacy BLE scan; gated by `neverForLocation` from API 31.

§4 — Crash + perf

No SDK in v1. The app is small (< 25 MB on both platforms) and the surface area is small (3 games + a lobby + connectivity). Logcat / Console.app are the only crash signals we have until v1.1.

§5 — Battery & data

- **Battery**: Wi-Fi is dominant; BLE is fine for hours. Mini Racer at 30 Hz is the heaviest workload and burns ~5–8% per hour on a Pixel 6.
- **Data**: zero cellular use. The app never reaches the internet.

§6 — Monetisation

None in v1. `PremiumGateService` is a stub returning `unlocked = true` unconditionally. v1.1 will: 1. Attach RevenueCat (iOS) + Google Play Billing (Android). 2. Gate Mini Racer behind a one-time IAP. 3. Optionally show ads in the lobby (cached AdMob, opt-out toggle).

Ad SDK is not pulled in for v1 to keep the .apk / .ipa under 25 MB.

§7 — App size budget

Surface	Target	Why
iOS .ipa	< 25 MB	Smaller than 30 MB so install over cellular doesn't prompt.
Android .aab	< 30 MB (download)	Same.

If we exceed the budget, the suspects are: (1) game assets (sprites, sounds), (2) ad SDK if added prematurely, (3) bundled fonts. Audit with `bundletool` on Android and Xcode's "Generate App Size Report" on iOS.

§8 — Cross-platform play matrix

See `CONNECTIVITY.md §7` for the matrix.

§9 — Watch tier

Phone-only in v1. Watches add ~30% file count for negative value here: both peers would need to be watches *and* paired phones, plus the screen is too small for any of the three games. Revisit in v2 only if there's clear demand.

§10 — Push / background

Neither platform requests notification permission in v1. There are no background modes — both peers must keep BuddyPlay foregrounded during play. v1.1 may add a "rejoin when back in range" hint via local notifications.

QUICKSTART

QUICKSTART.md

QUICKSTART

Prereqs

	macOS	Linux / Windows
iOS	Xcode 15+, `brew install xcodegen`	n/a
Android	JDK 17, Android SDK 34 (`ANDROID_HOME` set)	JDK 17, Android SDK 34
Marketing site	none — open `index.html` directly	none

60-second iOS spin-up (macOS)

```
git clone git@github.com:AmericanGroupLLC/BuddyPlay.git
cd BuddyPlay
brew install xcodegen
./scripts/run-ios-sim.sh
```

This generates the `ios/BuddyPlay.xcodeproj` , builds the Debug app, boots iPhone 15 Simulator, installs, and launches it.

60-second Android spin-up

```
cd BuddyPlay
./scripts/run-android-emulator.sh
```

Requires `ANDROID_HOME` and at least one created AVD (default name: `Pixel_6_API_34` ; override with `AVD_NAME=...`).

Pairing two real devices

1. Both phones on the **same Wi-Fi router** (or one running a hotspot the other has joined).
2. Phone A: open BuddyPlay → tap a game card → **Host** → note 4-char code.
3. Phone B: open BuddyPlay → tap **Join Nearby Game** → tap the host in the list → confirm code.
4. Game starts.

No internet required. If Wi-Fi is unavailable the lobby auto-falls back to BLE; turn-based games will still work, Mini Racer won't.

Dev workflow

```
## Edit shared logic in shared/BuddyCore/Sources/ ...
## Or shared logic mirror in android/core/src/main/ ...

## Run every test suite locally (skips iOS on non-macOS)
./scripts/test-all.sh

## Just iOS unit tests
cd shared/BuddyCore && swift test

## Just Android unit tests
cd android && ./gradlew :core:test :app:testDebugUnitTest

## Marketing site preview
python3 -m http.server   # then http://localhost:8000
```

Common gotchas

- **iOS Local Network prompt**: BuddyPlay surfaces it on the first Host/Join action, not at launch. Reject it once and the Settings → Privacy → Local Network toggle is the only way back.
- **Android BLE on API ≤30**: requires `ACCESS_FINE_LOCATION` even though we never collect location. Gated with `usesPermissionFlags="neverForLocation"` on API 31+ to skip the prompt entirely.
- **`:core` is JVM-only**: use `./gradlew :core:test`, NOT `:core:testDebugUnitTest`.
- **Same-version Bonjour**: both peers must be on the same `WireCodec` major version. v1 → v1.x is fine; v1 → v2 will toast “Update your friend’s app”.

RELEASE-VIDEO-SCRIPT

RELEASE-VIDEO-SCRIPT.md

Release Video Script — BuddyPlay

This file drives the silent MP4 release video built by the umbrella's `release-video.yml` reusable workflow at tag time.

Each `## Heading | <duration_seconds>` defines one scene. Within the scene, list keys as bullets. Recognised keys:

- `viewport: WIDTHxHEIGHT` (default 1280x720)
- `scroll: <px|%|top|bottom>` (default 0)
- `wait: <seconds|ms>` (dwell time after navigation)
- `caption: <text>` (placeholders `{APP_NAME}` and `{VERSION}`)
- `duration: <seconds>` (override the heading-level duration)

The intro and outro cards (3 s each) are auto-generated from the brand colour in `release.config.json`.

> **Tagline:** Play together, even when you're apart.

Hero — top of marketing site | 4

- viewport: 1280x720
- scroll: top
- wait: 600ms
- caption: {APP_NAME} {VERSION}

Features overview | 5

- viewport: 1280x720
- scroll: 25%
- wait: 600ms
- caption: New in {VERSION}

Highlight reel | 5

- viewport: 1280x720
- scroll: 50%
- wait: 600ms
- caption: Built for everyone

What's next | 5

- viewport: 1280x720
- scroll: 75%
- wait: 600ms

- caption: Available everywhere

Call to action — bottom of page | 4

- viewport: 1280x720
- scroll: bottom
- wait: 800ms
- caption: Try {APP_NAME} today

RELEASING

RELEASING.md

RELEASING.md

BuddyPlay's release pipeline is `release.yml`, gated on `pre-release-tests.yml`. Cutting a release is one tag push.

0. Pre-flight

- All keystone tests green locally: `./scripts/test-all.sh`
- Two-device manual smoke checklist passed (see `TESTING.md`).
- Marketing site preview reads correctly: `python3 -m http.server`.

1. Bump version

```
./scripts/bump-version.sh 1.2.0
git diff
```

This updates the iOS `Info.plist` and Android `:app:build.gradle.kts` versions to the new semver.

2. Commit + tag

```
git commit -am 'chore(release): v1.2.0'
git tag v1.2.0
git push origin main v1.2.0
```

Pushing the tag triggers `release.yml`. Pre-release tests must pass before any binary is built or uploaded.

3. What `release.yml` does

Job	Purpose	Gated on
`pre-release-tests`	Run every test suite (iOS unit + UI, Android unit + UI).	always
`build-android`	Build APK + AAB. Sign if `ANDROID_KEYSTORE_BASE64` secret set.	pre-release-tests
`build-ios`	Build unsigned `BuddyPlay.app` for iPhone Sim.	pre-release-tests
`build-web`	Zip the marketing site.	pre-release-tests
`publish-github-release`	Create the GitHub Release with all artefacts.	all build-*
`publish-play-store`	Upload AAB to Google Play.	secret `PLAY_STORE_SERVICE_ACCOUNT_JSON`
`publish-testflight`	Build signed `.ipa` + push to TestFlight.	secret `APP_STORE_CONNECT_API_KEY_P8_BASE64`

4. Required secrets

All optional — workflow no-ops gracefully when unset.

Secret	Purpose
`CODECOV_TOKEN`	Coverage upload (private repos).
`ANDROID_KEYSTORE_BASE64`	Sign Play Store AAB.
`ANDROID_KEYSTORE_PASSWORD`, `ANDROID_KEY_ALIAS`, `ANDROID_KEY_PASSWORD`	Sign Play Store AAB.
`PLAY_STORE_SERVICE_ACCOUNT_JSON`	Upload AAB to Play.
`PLAY_STORE_PACKAGE_NAME`	`com.americangroupllc.buddyplay`
`APP_STORE_CONNECT_API_KEY_ID`, `APP_STORE_CONNECT_API_ISSUER_ID`, `APP_STORE_CONNECT_API_KEY_P8_BASE64`	TestFlight upload.
`APPLE_TEAM_ID`	Code signing.
`MATCH_PASSWORD`, `MATCH_GIT_URL`	Fastlane Match (provisioning profile sync).

5. Pre-release tags

Use `-rc.N`, `-beta.N`, `-alpha.N` suffixes:

```
git tag v1.2.0-rc.1
```

These are flagged as pre-releases on GitHub and routed to the `beta` track on Google Play.

6. Hotfix

```
./scripts/bump-version.sh 1.2.1
git commit -am 'fix(chess): castling-through-check regression'
git tag v1.2.1
git push origin main v1.2.1
```

7. Local dry-run

Before tagging, validate the build will succeed:

```
./scripts/release-dry-run.sh v1.2.0
```

Stages all artefacts in `distribution/staging-v1.2.0/`. No tag is pushed.

SENTRY

SENTRY.md

SENTRY.md

TL;DR — Sentry is NOT enabled in v1.

BuddyPlay is offline-first. There is no Sentry DSN, no init call, no SDK binary in the v1 .ipa or .aab. This document is a forward-looking spec for v1.1 only.

Why we kept the stub

The `canImport`-gated `CrashReportingService` stub from the Drift / Card scaffold is preserved so v1.1 can wire Sentry up without an architecture refactor:

```
// shared/BuddyCore/Sources/BuddyCore/Observability/CrashReportingService.swift
public protocol CrashReportingService {
    func capture(error: Error, context: [String: String])
    func breadcrumb(_ message: String, category: String)
}

public final class NoopCrashReportingService: CrashReportingService {
    public init() {}
    public func capture(error: Error, context: [String: String]) {}
    public func breadcrumb(_ message: String, category: String) {}
}

#if canImport(Sentry)
import Sentry
public final class SentryCrashReportingService: CrashReportingService { ... }
#endif
```

Same shape on Android (`android/core/src/main/ … /observability/CrashReportingService.kt`).

v1.1 wiring plan (NOT in scope for v1)

1. Add **opt-in** Settings toggle: *"Send anonymous crash reports"*. Default OFF.
2. Behind that toggle, init Sentry with `tracesSampleRate = 0`, `enableUserInteractionTracing = false`, `attachScreenshot = false`.
3. Scrub PII: `beforeSend` strips opponent display names + peer UUIDs from the event payload.
4. Update `PRIVACY.md` to disclose what's sent and link to Sentry's privacy doc.
5. Set DSN via env var (CI secret), not hard-coded — same pattern Drift uses.

What stays no-op forever

- **Replays**: never. Our screen content includes opponent display names.
- **User identification**: never. Peer UUIDs are not user IDs and are not stable across devices.

- **Performance traces**: only via the Settings opt-in.

Why not in v1

- Adds ~2 MB to each platform binary.
- Requires a backend to receive events; we don't have one.
- Requires a privacy-policy update we don't want to ship as part of the initial release.

Where the no-SDK choice is enforced

- `release.yml` does NOT set `SENTRY_DSN_*` env vars in v1. (Slot exists for v1.1.)
- `ios/project.yml` does NOT add the Sentry SPM package.
- `android/app/build.gradle.kts` does NOT add the `io.sentry:sentry-android` dependency.
- `Info.plist` and `AndroidManifest.xml` do NOT request crash-related permissions or background modes.

SIGNING

SIGNING.md

Signing — BuddyPlay

This document lists every secret used by the release pipeline and the exact code-signing wiring for each platform. Workflows degrade gracefully when secrets are absent (Android → unsigned APK; iOS → unsigned Simulator `.app.zip`; Desktop → unsigned binaries).

See the canonical secret reference in the umbrella repo:
https://github.com/AmericanGroupLLC/AmericanGroupLLC/blob/main/SECRETS.md

---

Android

Secret	Required	Purpose
`ANDROID_KEYSTORE_BASE64`	optional	Base64-encoded `upload.jks`
`ANDROID_KEYSTORE_PASSWORD`	optional	Keystore password
`ANDROID_KEY_ALIAS`	optional	Key alias
`ANDROID_KEY_PASSWORD`	optional	Key password
`PLAY_STORE_SERVICE_ACCOUNT_JSON`	optional	Google Play upload
`PLAY_STORE_PACKAGE_NAME`	optional	Play Console package

A template is at `keystore.properties.example`.

---

iOS

The build-ios job takes one of two paths based on whether `APPLE_CERTIFICATE_BASE64` is set:

- **Set** → import .p12 to a temp keychain, install the provisioning profile, archive `-sdk iphoneos`, generate `exportOptions.plist` from `release.config.json.bundleId` (overridable via `APPLE_BUNDLE_ID`), `xcodebuild -exportArchive`, attach `<AppSlug>-iOS-<version>.ipa` to the GitHub Release.
- **Unset** → unsigned `-sdk iphonesimulator` build attached as `<AppSlug>-iOS-<version>-Simulator.app.zip` (Simulator-only, cannot run on real devices).

Secret	Required	Purpose
`APPLE_CERTIFICATE_BASE64`	optional	.p12 (Distribution)
`APPLE_CERTIFICATE_PASSWORD`	optional	.p12 password
`APPLE_KEYCHAIN_PASSWORD`	optional	Temp keychain pw
`APPLE_PROVISIONING_PROFILE_BASE64`	optional	Wildcard mobileprovision (recommended for (none))
`APPLE_TEAM_ID`	optional	10-char team ID
`APPLE_BUNDLE_ID`	optional	Override the default bundle id (`com.americangroupllc.buddyplay`)
`APP_STORE_CONNECT_API_KEY_ID`	optional	TestFlight upload
`APP_STORE_CONNECT_API_ISSUER_ID`	optional	TestFlight upload
`APP_STORE_CONNECT_API_KEY_P8_BASE64`	optional	TestFlight upload

Provisioning profile shape

The default bundle ID is `com.americangroupllc.buddyplay`. Because this app ships with extensions (`(none)`), use a **wildcard** profile such as `com.americangroupllc.buddyplay.*` so the profile covers the host app and every extension target. App Store Connect requires explicit IDs for each target, but the *upload step* is gated by a separate secret and only runs after the .ipa is built.

Desktop

The desktop build job runs on Ubuntu and produces:

- Windows NSIS installer `*.exe` (signed only if you add Windows Authenticode in a follow-up — out of scope here).
- Linux AppImage (unsigned, portable, no install required).
- macOS .dmg (**unsigned**; Gatekeeper will block on first launch. See `STORE-PACKAGING.md` for the `xattr -cr` bypass and the notarization roadmap).

---

watchOS

The release job builds watchOS as a **standalone Simulator .app.zip** for QA. Submitting a watch app to the App Store requires the watch target to be embedded inside the iOS `.ipa` (watch2-app pattern), which kicks in automatically once `APPLE_CERTIFICATE_BASE64` is set and the Xcode project's iOS target depends on the watch target.

STORE-PACKAGING

STORE-PACKAGING.md

STORE-PACKAGING.md

How BuddyPlay v1 ships to the App Store and Google Play.

§1 — App Store (iPhone)

Field	Value
Display name	BuddyPlay
Bundle ID	`com.americangroupllc.buddyplay`
Min OS	iOS 17
Devices	iPhone (no iPad-specific layout in v1)
Category	Games
Subcategory	Board / Card
Age rating	4+ (no UGC, no chat, no IAP, no ads)

Capabilities used

- **Bluetooth (always)** — `bluetooth-central` + `bluetooth-peripheral` background modes are NOT enabled in v1 (foreground play only).
- **Bonjour services** — declared in `Info.plist`'s `NSBonjourServices` array as `_buddyplay._tcp`.
- **Local Network usage** — `NSLocalNetworkUsageDescription` rationale string presented to user on first Host/Join action.

NOT used

- Push notifications (no APS entitlement).
- App Groups (single-app, single-process).
- HealthKit, Contacts, Camera, Microphone, Location, Photos.
- In-App Purchase (no StoreKit dependency in v1).
- Sign in with Apple (no accounts).
- Family Sharing — N/A (no IAPs).

Privacy nutrition label

Section	v1
Data Used to Track You	None
Data Linked to You	None
Data Not Linked to You	None

There is genuinely no data sent to any server.

App Review notes

> BuddyPlay is an offline multiplayer hub. To test the full experience, two devices on the same Wi-Fi network are required. A solo Demo Mode is available from Settings → Demo Match for review purposes; it spins up an in-process mock peer.

(The Demo Mode is NOT user-facing in production builds — it's behind an `#if DEBUG` gate plus a build-config flag flipped on for App Review only.)

§2 — Google Play (Android)

Field	Value
Application label	BuddyPlay
Package	`com.americangroupllc.buddyplay`
`minSdkVersion`	26 (Android 8)
`targetSdkVersion`	34 (Android 14)
Category	Games → Board
Content rating	Everyone
Ads	None
In-app purchases	None

Permissions declared

See `CONNECTIVITY.md §9`. Summary: Bluetooth scan/connect/advertise + Wi-Fi state + nearby Wi-Fi (API 33+) only. **No camera, no mic, no location on modern Android, no notifications.**

Data safety form

Question	Answer
Does your app collect or share any user data?	**No**
Does your app encrypt data in transit?	N/A (no internet traffic)
Can users request deletion?	N/A (no remote data; "Erase all rivalries" wipes local store)

Play Console release tracks

`release.yml` routes by tag suffix: - `vX.Y.Z` → production - `vX.Y.Z-rc.N` → beta - `vX.Y.Z-beta.N` → beta - `vX.Y.Z-alpha.N` → alpha - `vX.Y.Z-internal.N` → internal

§3 — Marketing assets

Per-store screenshots live in `distribution/screenshots/<platform>/<lang>/` (not committed; generated on demand from Simulator / Emulator). Required:

- iOS: 6.7″ (iPhone 16 Pro Max), 6.5″ (iPhone 11 Pro Max), 5.5″ (iPhone 8 Plus).
- Android: phone (1080×1920) — minimum 2, maximum 8.

§4 — What's new

`distribution/whatsnew/whatsnew-en-US/whatsnew.txt` is uploaded by `release.yml` as the Play Store release notes. Keep it under 500 chars.

§5 — Bundle size budget

See `PRODUCTION.md §7`.

§6 — Why no Watch / Wear tier

Out of scope for v1. Watch UX needs a partner watch *and* phone, and our target use case (two phones nearby) doesn't benefit. Revisit in v2 only on demand.

Desktop binaries (Electron)

The `build-desktop` job in `.github/workflows/release.yml` uses `electron-builder` on an Ubuntu runner to produce three artifacts attached to every GitHub Release:

Artifact	Signed?	Notes
`BuddyPlay-Setup-X.Y.Z.exe` (NSIS)	no	Windows SmartScreen will prompt on first run; click *More info* → *Run anyway*.
`BuddyPlay-X.Y.Z.AppImage`	no	`chmod +x` then double-click on Linux.
`BuddyPlay-X.Y.Z.dmg`	**no**	macOS Gatekeeper will refuse. To install:

```
xattr -cr "/Volumes/BuddyPlay/BuddyPlay.app"
cp -R "/Volumes/BuddyPlay/BuddyPlay.app" /Applications/
```

(or *System Settings* → *Privacy & Security* → *Open Anyway*.)

A future change can add a separate `macos-latest` job + Apple Developer cert to produce a properly notarised .dmg without restructuring this workflow.

The Electron shell loads the existing root `index.html` in a `BrowserWindow` with `contextIsolation` enabled. See `desktop/main.js`.

TESTING

TESTING.md

TESTING.md

Shape of the test suite

Layer	Where	Framework	Runs in CI
Apple shared logic	`shared/BuddyCore/Tests/BuddyCoreTests/`	XCTest	`ci.yml`, `ios.yml`, `pre-release-tests.yml`
iOS UI smoke	`ios/BuddyPlayUITests/`	XCUITest	`ios.yml`, `pre-release-tests.yml`
Android shared logic	`android/core/src/test/ …`	JUnit 4 + Truth	`ci.yml`, `pre-release-tests.yml`
Android app unit	`android/app/src/test/ …`	JUnit 4 + Truth	`ci.yml`, `pre-release-tests.yml`
Android UI smoke	`android/app/src/androidTest/ …`	Compose UI testing	`android.yml`, `pre-release-tests.yml`

The keystone tests (these are the project's safety net)

Every keystone helper has an XCTest **and** a JUnit twin. They test the same contract on both runtimes so behaviour cannot drift.

Helper	XCTest	JUnit	What it asserts
`HostElection`	`HostElectionTests`	`HostElectionKtTest`	Deterministic; symmetric (both peers compute the same host); UUID lex order tiebreak; platform tiebreak.
`WireCodec`	`WireCodecTests`	`WireCodecKtTest`	Round-trip `encode→decode==input` for every game's input + state. Rejects unknown schema versions.
`ChessRules`	`ChessRulesTests`	`ChessRulesKtTest`	Scholar's Mate detected. En passant captured. Castling blocked through check. Promotion to queen. ~30 cases.
`LudoRules`	`LudoRulesTests`	`LudoRulesKtTest`	6 grants extra turn. Capture sends opponent home. Only-6-leaves-base. Win on all-4-home.
`RacerPhysics`	`RacerPhysicsTests`	`RacerPhysicsKtTest`	Deterministic given fixed seed + fixed `dt`. Wall collision reflects velocity. Idle decay.
`LocalRivalryStore`	`LocalRivalryStoreTests`	`LocalRivalryStoreKtTest`	Write→read→increment yields right tallies. Corrupt JSON falls back to empty.

These six files MUST compile + test green on both Swift and Kotlin before any UI work for the corresponding game ships.

Running locally

```
## Everything (auto-skips suites the host can't run)
./scripts/test-all.sh

## Just BuddyCore
cd shared/BuddyCore && swift test
## or with coverage on iPhone Sim
xcodebuild test -scheme BuddyCore-Package \
  -destination 'platform=iOS Simulator,name=iPhone 15,OS=latest' \
  -enableCodeCoverage YES CODE_SIGNING_ALLOWED=NO

## Just Android :core
cd android && ./gradlew :core:test

## Just Android :app unit
cd android && ./gradlew :app:testDebugUnitTest

## Compose UI smoke (requires emulator + adb)
cd android && ./gradlew :app:connectedDebugAndroidTest

## Just XCUITest smoke
cd ios && xcodegen generate
xcodebuild test -project BuddyPlay.xcodeproj -scheme BuddyPlay \
  -sdk iphonesimulator -configuration Debug \
  -destination 'platform=iOS Simulator,name=iPhone 15,OS=latest' \
  CODE_SIGNING_ALLOWED=NO
```

Manual cross-platform / cross-transport checklist (pre-release)

Scenario	Steps	Pass criterion
iOS host ↔ Android guest, Chess, Wi-Fi	Both phones same router. iOS hosts Chess, Android joins. Make first move.	First move syncs in < 200 ms.
iOS host ↔ Android guest, Chess, BLE	Disable Wi-Fi on both phones. iOS hosts. Android joins via BLE.	First move syncs in < 1 s.
Hotspot fallback	Turn off router. iOS enables hotspot. Android joins it.	Chess game runs to completion.
Mini Racer rejects BLE	Disable Wi-Fi. iOS picks Mini Racer.	UI shows “Mini Racer needs Wi-Fi or Hotspot” and BLE is disabled.
Local rivalry tally	Play Chess → quit → play Chess again.	Rivalries screen shows 2 games against the same opponent.
Erase rivalries	Settings → Erase.	Rivalries screen empty state.
Reset device ID	Settings → Reset device ID → re-pair with friend.	Friend sees a *new* opponent (rivalries restart).

Coverage targets

- BuddyCore (Swift): 80%+ on the keystone helpers.
- :core (Kotlin): 80%+ on the keystone helpers.
- App layers: best-effort; these are mostly thin glue.

Coverage is uploaded by CI to Codecov; the report is informational, not a blocker.